

UNIVERSIDAD PRIVADA DOMINGO SAVIO

Diseño Web II

Entrega 5B

Editar y Eliminar Materias

FASE 2 — Gestión de Usuarios y Materias

Abril 2026

Proyecto: gestion-academica

Objetivos de Aprendizaje

Al completar esta entrega, el estudiante será capaz de:

- Implementar API Routes PUT y DELETE para completar un CRUD
- Usar rutas dinámicas con parámetros async ([id]) en Next.js 16
- Construir un formulario de edición que precarga datos existentes
- Reutilizar un Dialog modal tanto para crear como para editar
- Agregar una columna de acciones con botones Editar y Eliminar
- Implementar diálogos de confirmación para operaciones destructivas (DELETE)
- Manejar errores de integridad referencial (FK constraints) desde el servidor
- Aplicar el patrón de refactorización: componente único para crear/editar

Prerrequisitos

Antes de comenzar, asegúrate de haber completado:

- ✓ Entregas 1A a 4B completadas y funcionando
- ✓ Entrega 5A — API y Listado de Materias
- ✓ Al menos 3 materias creadas con el formulario de la Entrega 5A
- ✓ Al menos 1 docente activo asignado a alguna materia

i Importante: En esta entrega modificaremos el componente `materias-table.tsx` de la Entrega 5A. Se recomienda hacer una copia de respaldo antes de comenzar: `cp materias-table.tsx materias-table.backup.tsx`

Teoría: CRUD Completo y Refactorización de Componentes

CRUD — Las 4 Operaciones Básicas

Con esta entrega completaremos las 4 operaciones CRUD sobre la entidad materias:

Operación	Método HTTP	Entrega	Endpoint
Create (Crear)	POST	5A	/api/admin/materias
Read (Leer)	GET	5A	/api/admin/materias
Update (Editar)	PUT	5B (esta)	/api/admin/materias/[id]
Delete (Eliminar)	DELETE	5B (esta)	/api/admin/materias/[id]

PUT vs PATCH

Existen dos métodos HTTP para actualizar recursos:

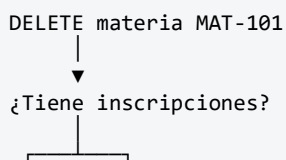
Método	Semántica	Uso en nuestro proyecto
PUT	Reemplaza todo el recurso	Enviamos TODOS los campos de la materia
PATCH	Actualiza parcialmente	Enviaríamos solo los campos que cambiaron

i Decisión pedagógica: Usamos PUT porque es más simple: el formulario de edición siempre envía todos los campos. PATCH es más eficiente en APIs profesionales, pero agrega complejidad innecesaria para este curso.

Integridad Referencial al Eliminar

Cuando intentamos eliminar una materia que tiene inscripciones asociadas, PostgreSQL lanza un error de FK constraint. Debemos manejar este caso:

Diagrama: Restricción FK al eliminar



```

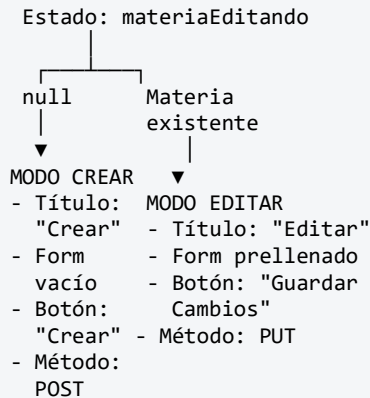
  Sí      NO
  ↓      ↓
Error    Eliminada
23503    exitosamente
  ↓
"No se puede eliminar:
tiene inscripciones"
```

⚠ Cascada: Nuestro esquema NO usa ON DELETE CASCADE en inscripciones. Esto es intencional: no queremos que al eliminar una materia se borren automáticamente las inscripciones de los estudiantes. Preferimos manejar el error explícitamente.

Patrón: Dialog Reutilizable (Crear + Editar)

En lugar de crear dos Dialogs separados (uno para crear y otro para editar), refactorizaremos el componente para usar un solo Dialog que funcione en ambos modos:

Patrón: Dialog dual



☑ **Reutilización:** Este patrón reduce la duplicación de código. Un solo Dialog, un solo formData, una función guardarMateria() que decide si hacer POST o PUT según si materiaEditando es null o no.

Paso 1 — API Route: Editar Materia (PUT)

Crearemos el endpoint para actualizar una materia existente. La ruta usa [id] como parámetro dinámico.

1.1 — Crear la estructura

Terminal

```
mkdir -p "app/api/admin/materias/[id]"
```

1.2 — Crear la API Route

Crea el archivo `app/api/admin/materias/[id]/route.ts`:

app/api/admin/materias/[id]/route.ts

```
import { createClient } from "@lib/supabase/server";
import { createClient as createServiceClient } from "@supabase/supabase-js";
import { NextRequest, NextResponse } from "next/server";
import { materiaSchema } from "@lib/validations/materia";

// ===== PUT: Editar materia =====
export async function PUT(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params;

    // 1. Verificar autenticación y rol admin
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "admin") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    // 2. Validar body con Zod
```

```

const body = await request.json();
const resultado = materiaSchema.safeParse(body);

if (!resultado.success) {
  const errores = resultado.error.issues.map((issue) => ({
    campo: issue.path.join("."),
    mensaje: issue.message,
  }));
  return NextResponse.json(
    { error: "Datos inválidos", errores },
    { status: 400 }
  );
}

// 3. Actualizar con Service Role
const supabaseAdmin = createServiceClient(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.SUPABASE_SERVICE_ROLE_KEY!
);

const { data, error } = await supabaseAdmin
  .from("materias")
  .update({
    codigo: resultado.data.codigo,
    nombre: resultado.data.nombre,
    creditos: resultado.data.creditos,
    semestre: resultado.data.semestre,
    docente_id: resultado.data.docente_id || null,
  })
  .eq("id", id)
  .select(`
    id, codigo, nombre, creditos, semestre, docente_id,
    docente:usuarios_perfil!docente_id(id, nombre, apellido)
  `)
  .single();

if (error) {
  console.error("Error al actualizar materia:", error);

  if (error.code === "23505") {
    return NextResponse.json(
      { error: "Ya existe otra materia con ese código" },
      { status: 409 }
    );
  }

  return NextResponse.json(
    { error: "Error al actualizar la materia" },
    { status: 500 }
  );
}

return NextResponse.json({ materia: data });
} catch (err) {
  console.error("Error inesperado:", err);
  return NextResponse.json(
    { error: "Error interno del servidor" },
    { status: 500 }
  );
}
}

```

1.3 — Comparación POST vs PUT

Observa las similitudes y diferencias entre POST (crear, Entrega 5A) y PUT (editar):

Aspecto	POST (Crear)	PUT (Editar)
Operación Supabase	<code>.insert()</code>	<code>.update().eq("id", id)</code>
Parámetro id	No necesita	Viene de await params
Status éxito	201 (Created)	200 (OK)
Error 23505	"Ya existe una materia"	"Ya existe otra materia"
Validación Zod	Igual: <code>safeParse(body)</code>	Igual: <code>safeParse(body)</code>
Protección	Igual: auth + rol admin	Igual: auth + rol admin

☒ **Reutilización del schema:** Usamos exactamente el mismo `materiaSchema` de la Entrega 5A. El PUT valida los datos con las mismas reglas que el POST.

Paso 2 — API Route: Eliminar Materia (DELETE)

Agregamos el handler DELETE en el mismo archivo [id]/route.ts. Esta operación debe manejar el caso de integridad referencial.

2.1 — Agregar el handler DELETE

Abre app/api/admin/materias/[id]/route.ts y agrega después de la función PUT:

app/api/admin/materias/[id]/route.ts – agregar después del PUT

```
// ===== DELETE: Eliminar materia =====
export async function DELETE(
  request: NextRequest,
  { params }: { params: Promise<{ id: string }> }
) {
  try {
    const { id } = await params;

    // 1. Verificar autenticación y rol admin
    const supabase = await createClient();
    const {
      data: { user },
      error: authError,
    } = await supabase.auth.getUser();

    if (authError || !user) {
      return NextResponse.json({ error: "No autenticado" }, { status: 401 });
    }

    const { data: perfil } = await supabase
      .from("usuarios_perfil")
      .select("rol")
      .eq("id", user.id)
      .single();

    if (perfil?.rol !== "admin") {
      return NextResponse.json({ error: "No autorizado" }, { status: 403 });
    }

    // 2. Eliminar con Service Role
    const supabaseAdmin = createServiceClient(
      process.env.NEXT_PUBLIC_SUPABASE_URL!,
      process.env.SUPABASE_SERVICE_ROLE_KEY!
    );

    const { error } = await supabaseAdmin
      .from("materias")
      .delete()
      .eq("id", id);

    if (error) {
      console.error("Error al eliminar materia:", error);

      // Error de FK: tiene inscripciones asociadas
    }
  }
}
```

```

    if (error.code === "23503") {
      return NextResponse.json(
        {
          error:
            "No se puede eliminar esta materia porque tiene inscripciones asociadas. Primero
elimina las inscripciones.",
        },
        { status: 409 }
      );
    }

    return NextResponse.json(
      { error: "Error al eliminar la materia" },
      { status: 500 }
    );
  }

  return NextResponse.json({ mensaje: "Materia eliminada exitosamente" });
} catch (err) {
  console.error("Error inesperado:", err);
  return NextResponse.json(
    { error: "Error interno del servidor" },
    { status: 500 }
  );
}
}
}

```

2.2 — Códigos de error de PostgreSQL

Es importante conocer los códigos de error de PostgreSQL que manejamos en nuestras API Routes:

Código PG	Nombre	Cuándo ocurre	Nuestro manejo
23505	unique_violation	INSERT/UPDATE con valor duplicado en columna UNIQUE	Status 409: "Ya existe una materia con ese código"
23503	foreign_key_violation	DELETE cuando hay registros hijos que dependen del registro	Status 409: "Tiene inscripciones asociadas"
23502	not_null_violation	INSERT/UPDATE con NULL en columna NOT NULL	Status 500 genérico (Zod previene esto)

i Sin body: El handler DELETE no necesita leer el body del request. Solo usa el id del parámetro de ruta. El DELETE de Supabase tampoco devuelve datos por defecto, así que solo verificamos si hubo error.

Paso 3 — Refactorizar el Componente MateriasTable

Ahora actualizaremos el componente `materias-table.tsx` de la Entrega 5A para agregar: modo edición en el Dialog, columna de acciones, y botón de eliminar con confirmación.

⚠ Archivo completo: Este paso reemplaza completamente el archivo `materias-table.tsx`. Recomendamos copiar el archivo nuevo completo en lugar de intentar editar parcialmente el existente.

3.1 — Sección A: Imports y tipos (actualizada)

Reemplaza completamente `components/dashboard/admin/materias-table.tsx`:

```
components/dashboard/admin/materias-table.tsx — Parte 1/6
"use client";

import { useEffect, useState } from "react";
import { toast } from "sonner";

// Componentes shadcn/ui
import {
  Table, TableBody, TableCell, TableHead,
  TableHeader, TableRow,
} from "@components/ui/table";
import { Badge } from "@components/ui/badge";
import { Button } from "@components/ui/button";
import {
  Select, SelectContent, SelectItem,
  SelectTrigger, SelectValue,
} from "@components/ui/select";
import {
  Dialog, DialogContent, DialogDescription,
  DialogFooter, DialogHeader, DialogTitle,
  DialogTrigger,
} from "@components/ui/dialog";
import {
  AlertDialog, AlertDialogAction, AlertDialogCancel,
  AlertDialogContent, AlertDialogDescription,
  AlertDialogFooter, AlertDialogHeader,
  AlertDialogTitle, AlertDialogTrigger,
} from "@components/ui/alert-dialog";
import { Input } from "@components/ui/input";
import { Label } from "@components/ui/label";

// Iconos
import { Plus, RefreshCw, Pencil, Trash2 } from "lucide-react";

// Tipos
interface Docente {
  id: string;
  nombre: string;
  apellido: string;
}
```

```
}  
  
interface Materia {  
  id: string;  
  codigo: string;  
  nombre: string;  
  creditos: number;  
  semestre: string;  
  docente_id: string | null;  
  created_at: string;  
  docente: Docente | null;  
}
```

i Nuevos imports: Agregamos AlertDialog (para confirmar eliminación) y los iconos Pencil y Trash2 (para los botones de acción en cada fila).

3.2 — Sección B: Estado (actualizado)

```
components/dashboard/admin/materias-table.tsx — Parte 2/6
// Estado inicial del formulario
const formInicial = {
  codigo: "",
  nombre: "",
  creditos: 4,
  semestre: "",
  docente_id: "",
};

export default function MateriasTable() {
  // Estados de datos
  const [materias, setMaterias] = useState<Materia[]>([]);
  const [docentes, setDocentes] = useState<Docente[]>([]);
  const [loading, setLoading] = useState(true);

  // Filtro
  const [filtroSemestre, setFiltroSemestre] = useState<string>("todos");

  // Dialog crear/editar
  const [dialogOpen, setDialogOpen] = useState(false);
  const [guardando, setGuardando] = useState(false);
  const [formData, setFormData] = useState(formInicial);

  // NUEVO: materia en edición (null = modo crear)
  const [materiaEditando, setMateriaEditando] = useState<Materia | null>(null);

  // Funciones de carga (sin cambios)
  const cargarMaterias = async () => {
    setLoading(true);
    try {
      const res = await fetch("/api/admin/materias");
      const data = await res.json();
      if (!res.ok) {
        toast.error(data.error || "Error al cargar materias");
        return;
      }
      setMaterias(data.materias);
    } catch {
      toast.error("Error de conexión");
    } finally {
      setLoading(false);
    }
  };

  const cargarDocentes = async () => {
    try {
      const res = await fetch("/api/admin/docentes");
      const data = await res.json();
      if (res.ok) setDocentes(data.docentes);
    } catch {
      console.error("Error al cargar docentes");
    }
  };

  useEffect(() => {
    cargarMaterias();
    cargarDocentes();
  }, []);
}
```

```
const semestres = [...new Set(materias.map((m) => m.semestre))].sort();  
const materiasFiltradas = materias.filter(  
  (m) => filtroSemestre === "todos" || m.semestre === filtroSemestre  
);
```

La variable clave es `materiaEditando`: cuando es `null` estamos en modo crear; cuando contiene una materia, estamos en modo editar.

3.3 — Sección C: Funciones de acción (nuevas)

components/dashboard/admin/materias-table.tsx — Parte 3/6

```
// Abrir Dialog en modo CREAR
const abrirCrear = () => {
  setMateriaEditando(null);
  setFormData(formInicial);
  setDialogOpen(true);
};

// Abrir Dialog en modo EDITAR
const abrirEditar = (materia: Materia) => {
  setMateriaEditando(materia);
  setFormData({
    codigo: materia.codigo,
    nombre: materia.nombre,
    creditos: materia.creditos,
    semestre: materia.semestre,
    docente_id: materia.docente_id || "",
  });
  setDialogOpen(true);
};

// Guardar materia (POST o PUT según el modo)
const guardarMateria = async () => {
  setGuardando(true);
  try {
    const esEdicion = materiaEditando !== null;
    const url = esEdicion
      ? `/api/admin/materias/${materiaEditando.id}`
      : "/api/admin/materias";
    const method = esEdicion ? "PUT" : "POST";

    const res = await fetch(url, {
      method,
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        ...formData,
        creditos: Number(formData.creditos),
        docente_id: formData.docente_id || null,
      }),
    });

    const data = await res.json();

    if (!res.ok) {
      if (data.errores) {
        data.errores.forEach(
          (e: { campo: string; mensaje: string }) =>
            toast.error(`${e.campo}: ${e.mensaje}`)
        );
      } else {
        toast.error(data.error || "Error al guardar materia");
      }
      return;
    }

    toast.success(
      esEdicion
        ? `Materia "${data.materia.codigo}" actualizada`
        : `Materia "${data.materia.codigo}" creada`
    );
  }
};
```

```
);  
setDialogOpen(false);  
setFormData(formInicial);  
setMateriaEditando(null);  
cargarMaterias();  
} catch {  
  toast.error("Error de conexión");  
} finally {  
  setGuardando(false);  
}  
};
```

☑ **Un solo handler:** guardarMateria() decide el método y URL según el estado de materiaEditando. No necesitamos dos funciones separadas.

3.4 — Sección D: Función eliminar (nueva)

```
components/dashboard/admin/materias-table.tsx — Parte 4/6
// Eliminar materia
const eliminarMateria = async (materia: Materia) => {
  try {
    const res = await fetch(`/api/admin/materias/${materia.id}`, {
      method: "DELETE",
    });

    const data = await res.json();

    if (!res.ok) {
      toast.error(data.error || "Error al eliminar materia");
      return;
    }

    toast.success(`Materia "${materia.codigo}" eliminada`);
    cargarMaterias();
  } catch {
    toast.error("Error de conexión");
  }
};
```

La función `eliminarMateria` es llamada desde el `AlertDialog` de confirmación. Solo se ejecuta cuando el usuario hace clic en "Confirmar".

3.5 — Sección E: Renderizado — Barra superior y Dialog (actualizado)

El Dialog ahora cambia su título y botón según el modo:

components/dashboard/admin/materias-table.tsx — Parte 5/6

```
return (
  <div className="space-y-4">
    {/* ===== BARRA SUPERIOR ===== */}
    <div className="flex items-center gap-4">
      <Select value={filtroSemestre} onChange={setFiltroSemestre}>
        <SelectTrigger className="w-[200px]">
          <SelectValue placeholder="Filtrar por semestre" />
        </SelectTrigger>
        <SelectContent>
          <SelectItem value="todos">Todos los semestres</SelectItem>
          {semestres.map((s) => (
            <SelectItem key={s} value={s}>{s}</SelectItem>
          ))}
        </SelectContent>
      </Select>

      <Button variant="outline" size="sm" onClick={cargarMaterias}>
        <RefreshCw className="mr-2 h-4 w-4" /> Refrescar
      </Button>

      <span className="text-sm text-muted-foreground">
        {materiasFiltradas.length} materia(s)
      </span>

      <div className="ml-auto">
        <Button onClick={abrirCrear}>
          <Plus className="mr-2 h-4 w-4" /> Nueva Materia
        </Button>
      </div>
    </div>

    {/* ===== DIALOG CREAR/EDITAR ===== */}
    <Dialog open={dialogOpen} onOpenChange={({open}) => {
      setDialogOpen(open);
      if (!open) {
        setMateriaEditando(null);
        setFormData(formInicial);
      }
    }}>
      <DialogContent className="sm:max-w-[500px]">
        <DialogHeader>
          <DialogTitle>
            {materiaEditando ? "Editar Materia" : "Crear Nueva Materia"}
          </DialogTitle>
          <DialogDescription>
            {materiaEditando
              ? `Editando: ${materiaEditando.codigo} - ${materiaEditando.nombre}`
              : "Completa los datos de la materia. El código debe ser único."}
          </DialogDescription>
        </DialogHeader>

        <div className="grid gap-4 py-4">
          {/* Código */}
          <div className="grid gap-2">
```

```

<Label htmlFor="codigo">Código</Label>
<Input
  id="codigo"
  placeholder="MAT-101"
  value={formData.codigo}
  onChange={(e) =>
    setFormData({ ...formData, codigo: e.target.value.toUpperCase() })
  }
/>
</div>

{/* Nombre */}
<div className="grid gap-2">
  <Label htmlFor="nombre">Nombre</Label>
  <Input
    id="nombre"
    placeholder="Cálculo Diferencial"
    value={formData.nombre}
    onChange={(e) =>
      setFormData({ ...formData, nombre: e.target.value })
    }
  />
</div>

{/* Créditos y Semestre */}
<div className="grid grid-cols-2 gap-4">
  <div className="grid gap-2">
    <Label htmlFor="creditos">Créditos</Label>
    <Input
      id="creditos" type="number" min={1} max={10}
      value={formData.creditos}
      onChange={(e) =>
        setFormData({ ...formData, creditos: parseInt(e.target.value) || 1 })
      }
    />
  </div>
  <div className="grid gap-2">
    <Label htmlFor="semestre">Semestre</Label>
    <Input
      id="semestre" placeholder="2026-1"
      value={formData.semestre}
      onChange={(e) =>
        setFormData({ ...formData, semestre: e.target.value })
      }
    />
  </div>
</div>

{/* Docente */}
<div className="grid gap-2">
  <Label>Docente (opcional)</Label>
  <Select
    value={formData.docente_id || "sin-asignar"}
    onChange={(v) =>
      setFormData({ ...formData, docente_id: v === "sin-asignar" ? "" : v })
    }
  >
    <SelectTrigger>
      <SelectValue placeholder="Seleccionar docente" />
    </SelectTrigger>
    <SelectContent>
      <SelectItem value="sin-asignar">Sin asignar</SelectItem>
      {docentes.map((d) => (

```

```

        <SelectItem key={d.id} value={d.id}>
            {d.nombre} {d.apellido}
        </SelectItem>
    ))}
</SelectContent>
</Select>
</div>
</div>

<DialogFooter>
    <Button variant="outline" onClick={() => setDialogOpen(false)}>
        Cancelar
    </Button>
    <Button onClick={guardarMateria} disabled={guardando}>
        {guardando
            ? "Guardando..."
            : materiaEditando ? "Guardar Cambios" : "Crear Materia"}
    </Button>
</DialogFooter>
</DialogContent>
</Dialog>

```

i onOpenChange: Cuando el Dialog se cierra (open = false), limpiamos materiaEditando y formData. Esto asegura que al abrir "Nueva Materia" el form siempre esté limpio.

3.6 — Sección F: Renderizado — Tabla con columna de acciones

components/dashboard/admin/materias-table.tsx — Parte 6/6

```

{ /* ===== TABLA ===== */}
<div className="rounded-md border">
  <Table>
    <TableHeader>
      <TableRow>
        <TableHead>Código</TableHead>
        <TableHead>Nombre</TableHead>
        <TableHead>Créditos</TableHead>
        <TableHead>Semestre</TableHead>
        <TableHead>Docente</TableHead>
        <TableHead className="text-right">Acciones</TableHead>
      </TableRow>
    </TableHeader>
    <TableBody>
      {loading ? (
        <TableRow>
          <TableCell colSpan={6} className="text-center py-8">
            Cargando materias...
          </TableCell>
        </TableRow>
      ) : materiasFiltradas.length === 0 ? (
        <TableRow>
          <TableCell colSpan={6} className="text-center py-8">
            No se encontraron materias
          </TableCell>
        </TableRow>
      ) : (
        materiasFiltradas.map((m) => (
          <TableRow key={m.id}>
            <TableCell className="font-mono font-medium">
              {m.codigo}
            </TableCell>
            <TableCell>{m.nombre}</TableCell>
            <TableCell>
              <Badge variant="secondary">{m.creditos}</Badge>
            </TableCell>
            <TableCell>
              <Badge variant="outline">{m.semestre}</Badge>
            </TableCell>
            <TableCell>
              {m.docente
                ? `${m.docente.nombre} ${m.docente.apellido}`
                : <span className="text-muted-foreground italic">Sin asignar</span>
              }
            </TableCell>

            { /* NUEVA: Columna de acciones */}
            <TableCell className="text-right">
              <div className="flex justify-end gap-2">

                { /* Botón Editar */}
                <Button
                  variant="ghost"
                  size="sm"
                  onClick={() => abrirEditar(m)}
                >
                  <Pencil className="h-4 w-4" />
                </Button>

                { /* Botón Eliminar con confirmación */}

```

```

<AlertDialog>
  <AlertDialogTrigger asChild>
    <Button
      variant="ghost"
      size="sm"
      className="text-destructive hover:text-destructive"
    >
      <Trash2 className="h-4 w-4" />
    </Button>
  </AlertDialogTrigger>
  <AlertDialogContent>
    <AlertDialogHeader>
      <AlertDialogTitle>
        ¿Eliminar materia?
      </AlertDialogTitle>
      <AlertDialogDescription>
        Estás a punto de eliminar la materia{" "}
        <strong>{m.codigo} - {m.nombre}</strong>.
        Esta acción no se puede deshacer.
      </AlertDialogDescription>
    </AlertDialogHeader>
    <AlertDialogFooter>
      <AlertDialogCancel>
        Cancelar
      </AlertDialogCancel>
      <AlertDialogAction
        onClick={() => eliminarMateria(m)}
        className="bg-destructive text-destructive-foreground hover:bg-
destructive/90"
      >
        Eliminar
      </AlertDialogAction>
    </AlertDialogFooter>
  </AlertDialogContent>
</AlertDialog>

</div>
</TableCell>
</TableRow>
))
}}
</TableBody>
</Table>
</div>
</div>
);
}

```

3.7 — Anatomía de los botones de acción

Botón	Variante	Comportamiento	Confirmación
Editar (Pencil)	ghost + sm	Abre Dialog en modo edición con datos precargados	No — editar no es destructivo
Eliminar (Trash2)	ghost + sm + destructive color	Muestra AlertDialog con nombre de la materia	Sí — AlertDialog antes de ejecutar

☑ **ghost variant:** Usamos `variant="ghost"` para que los botones sean sutiles y no compitan visualmente con los datos de la tabla. Solo el color rojo del icono de eliminar indica peligro.

Resumen de Archivos

En esta entrega creaste o modificaste los siguientes archivos:

Archivo	Acción	Descripción
app/api/admin/materias/[id]/route.ts	Nuevo	PUT: editar materia + DELETE: eliminar materia
components/dashboard/admin/materias-table.tsx	Modificado (reemplazo completo)	Dialog dual crear/editar + columna acciones + eliminar con AlertDialog

Archivos sin cambios que se reutilizan de la Entrega 5A:

Archivo	Reutilizado para
lib/validations/materia.ts	Validación Zod en el PUT (misma que en POST)
app/api/admin/materias/route.ts	GET y POST siguen funcionando sin cambios
app/api/admin/docentes/route.ts	Select de docentes en el formulario de edición
app/(dashboard)/admin/materias/page.tsx	Importa MateriasTable (sin cambios necesarios)

Verificación Final

Ejecuta `npm run dev` y verifica cada uno de estos puntos:

A. Tabla actualizada

- ✓ La tabla ahora tiene 6 columnas (se agregó Acciones)
- ✓ Cada fila muestra iconos de lápiz (editar) y papelera (eliminar)
- ✓ Los iconos son sutiles (variant ghost) y no distraen

B. Crear materia (debe seguir funcionando)

- ✓ Clic en "Nueva Materia" abre el Dialog con título "Crear Nueva Materia"
- ✓ El formulario está vacío y el botón dice "Crear Materia"
- ✓ Crear una materia funciona igual que en la Entrega 5A

C. Editar materia

- ✓ Clic en el icono lápiz abre el Dialog con título "Editar Materia"
- ✓ La descripción muestra el código y nombre de la materia
- ✓ Todos los campos están prellenados con los datos actuales
- ✓ El Select de docente muestra el docente actual seleccionado
- ✓ El botón dice "Guardar Cambios"
- ✓ Modificar un campo y guardar muestra toast de éxito
- ✓ La tabla se actualiza con los datos editados
- ✓ Cambiar el código a uno que ya existe muestra error 409

D. Eliminar materia

- ✓ Clic en el icono papelera abre un AlertDialog de confirmación
- ✓ El diálogo muestra el código y nombre de la materia a eliminar
- ✓ El botón "Eliminar" es rojo (destructive)
- ✓ Cancelar cierra el diálogo sin hacer nada
- ✓ Confirmar elimina la materia y muestra toast de éxito

- ✓ La tabla se actualiza sin la materia eliminada

E. Error de FK al eliminar

- ✓ Si en el futuro una materia tiene inscripciones, al intentar eliminarla muestra un mensaje claro de error

F. Dialog limpio

- ✓ Cerrar el Dialog de edición y luego abrir "Nueva Materia" muestra el form vacío
- ✓ Cerrar con la X también limpia el estado

Ejercicio Propuesto

Practica lo aprendido con estas tareas adicionales:

1. Soft delete en lugar de hard delete: En lugar de eliminar físicamente la materia (DELETE FROM), agrega una columna "activa" (BOOLEAN DEFAULT true) a la tabla materias. Modifica el endpoint DELETE para que haga UPDATE activa = false. En la tabla, muestra las materias inactivas con un Badge gris y un botón para reactivarlas.
2. Historial de cambios: Crea una tabla auditoria_materias con columnas (id, materia_id, campo_modificado, valor_anterior, valor_nuevo, modificado_por, fecha). En el endpoint PUT, antes de actualizar, guarda en esta tabla los campos que cambiaron. Muestra un botón "Ver historial" que despliegue los cambios de cada materia.
3. Validación con React Hook Form: Refactoriza el formulario del Dialog para usar React Hook Form + zodResolver con el materiaSchema, igual que hiciste con el perfil en la Entrega 4A. Instala el componente Form de shadcn si aún no lo tienes. Muestra errores inline bajo cada campo.
4. Acción masiva: Agrega checkboxes a cada fila de la tabla. Cuando el admin selecciona múltiples materias, muestra un botón "Eliminar seleccionadas" que ejecuta las eliminaciones en secuencia. Usa Promise.allSettled() para manejar errores individuales.

i Para entregar: Implementa al menos 1 de los 4 ejercicios. El ejercicio 3 es el más recomendado porque refuerza el patrón de validación dual visto en la teoría.

Próximo: Entrega 5C

En la siguiente entrega crearemos las vistas de materias específicas por rol:

- Vista docente: "Mis Materias" — lista de materias asignadas al docente logueado
- Vista estudiante: "Materias Disponibles" — catálogo de materias para inscripción
- Componente reutilizable MateriaCard para mostrar materias en formato card
- API Routes específicas por rol (GET materias filtradas por docente_id)
- Preparación para la Fase 3 de inscripciones

☒ **Preparación:** Asegúrate de tener al menos 5 materias creadas, con al menos 3 asignadas a un docente específico. Necesitarás iniciar sesión con ese docente para ver su vista "Mis Materias".

Fin de la Entrega 5B

Cierre de la Fase 2 — Gestión de Usuarios y Materias (entregas 4A—5B completadas)